

Aries : repenser l'infrastructure cloud pour les systèmes de serving agentiques

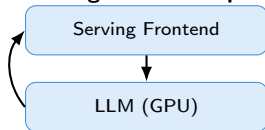
August 11, 2026

- 1 Contexte et motivation
- 2 Le framework Aries
- 3 Résultats expérimentaux
- 4 Vision : serving agent-native

Contexte et motivation

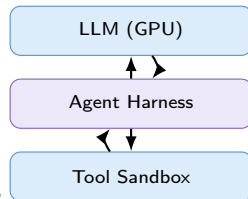
Du serving LLM classique au serving d'agents

Serving LLM classique



- Unité : la requête modèle isolée
- KV-cache libéré après la requête
- Métriques : TTFT, TPOT, débit de tokens

Serving agentique



- Unité : la trajectoire complète
- Contexte persistant sur toute la trajectoire
- Appels d'outils exécutés dans un sandbox isolé

Constat central

Chaque appel modèle n'est qu'une étape d'une boucle. Le serving agentique dépend autant du contexte et des outils que de l'inférence.

Le problème : un gap de mesure

- **Benchmarks capacité** (GAIA, SWE-bench...) : succès de la tâche, exécution en boîte noire.
- **Benchmarks systèmes** (MLPerf...) : performance système, mais à l'échelle d'une requête isolée.

Aucun des deux ne relie

progression de la tâche *et* comportement système inter-couches

Trois obstacles :

- 1 Sémantique de tâche imbriquée dans le harnais
- 2 Composants gérés indépendamment
- 3 Outils *stateful* et dépendants du substrat

Le framework Aries



Architecture d'Aries

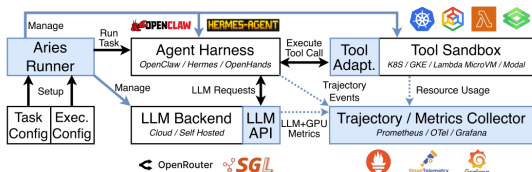


Figure: Architecture Aries

Trois défis, trois réponses

1. Préserver la sémantique de tâche

Task spec (charge, succès) séparée de l'**execution spec** (harnais, backend, sandbox).

2. Reconstruire les trajectoires

Identifiants de trajectoire propagés à travers modèle et outils, schéma d'événement commun.

3. Rendre les outils observables

Interface standardisée (setup, invocation, résultats) au-dessus d'adaptateurs par substrat.

Harnais intégrés

- OpenHands
- Hermes Agent
- OpenClaw

Benchmarks

- SWE-Bench Pro
- Terminal-Bench 2
- DeepResearch Bench

Modèle et infrastructure

- Qwen3.6-35B-A3B-FP8
- Servi localement via SGLang
- Nœud 96 cœurs, 1× NVIDIA H100 (94 Go HBM)

Données de production

- Trace de 8h, 10 instances LLM, 100 sandboxes

Méthode

20 tâches/benchmark, taux de succès 50% avec OpenClaw, 5 répétitions.

Résultats expérimentaux



Constat 1 : le harnais et les outils comptent autant que le modèle

Part de latence "outil"	OpenHands	Hermes	OpenClaw
Terminal-Bench 2	26%	48%	43%
DeepResearch Bench	30%	38%	13%
SWE-Bench Pro	37%	33%	29%

- Outils : jusqu'à **48%** de la latence totale.
- Harnais : jusqu'à **9%** d'orchestration en plus.
- Appels d'outils à **longue traîne**.

À retenir

La durée de bout en bout dépend fortement du harnais et du sandbox, pas seulement de l'inférence.

Constat 2 : contexte long, efficacité & précision (1/2)

- **Amplification de l'état** : budget fixe (25 Go) $\rightarrow$ capacité résidente 176 $\rightarrow$ 40 requêtes ($-4,4\times$).
- **Seuil de suffisance** : au-delà d'un seuil, plus d'historique n'aide plus.
 - OpenClaw / DeepResearch : pic à 95% à 160K tokens.
 - Terminal-Bench 2, SWE-Bench : plateau avant le max.

Constat 2 : contexte long, efficacité & précision (2/2)

- **Batching limité par la capacité** : pool à 97% utilisé, mais 22 requêtes concurrentes en médiane (32 en attente).
- Délai de file p_{95} : **7,9 s**.

À retenir

Retenir du contexte au-delà de son utilité réduit la capacité sans gain de précision.

Constat 3 : les sandboxes doivent être élastiques

- CPU/mémoire inactifs **80%** / **50%** du temps, avec pics brefs.

Keepalive	Effet coût
0s (arrêt immédiat)	Instance -65% , C/R $\times 4,9$
∞ (persistant)	Référence
Système idéal	$-3\times$ (hypothèse)

À retenir

Le snapshot capture/restore force un choix impossible entre idle payé et C/R répété.

Constat 4 : les sandboxes sont sous siège continu

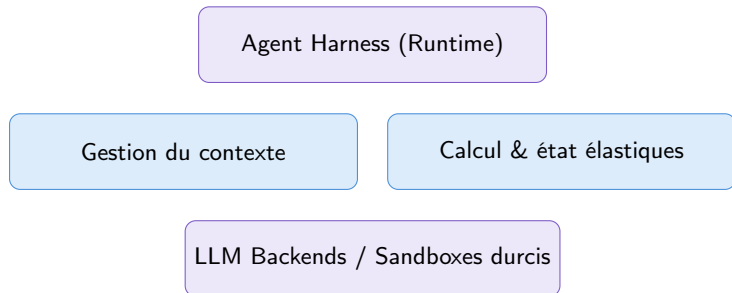
- Les agents IA découvrent déjà des vulnérabilités de façon autonome.
- Proxy : part des CVE Linux trouvées par des outils IA.
- **T2 2026** : $\times 3,5$ de CVE publiées vs records précédents.
- Exploiter une CVE connue devient **peu coûteux**.

À retenir

Les sandboxes doivent minimiser leur surface d'attaque et miser sur la défense en profondeur.

Vision : serving agent-native

Quatre axes de co-conception



- 1 **Métriques capturant la trajectoire** sur toutes les couches
- 2 **Plan de contrôle** pour la gestion de contexte consciente de la trajectoire
- 3 **Gestion élastique** de l'exécution d'outils via contrôle du sandbox
- 4 **Minimisation de la surface d'attaque** des sandboxes

- Signaux de progression en ligne, dérivés des événements de trajectoire.
- Rétention adaptative : compaction déclenchée quand le gain de précision ne justifie plus le coût mémoire.
- Placement hiérarchique de l'état (accélérateur / mémoire moins chère).

- Scaling vertical piloté par des signaux modèle/harnais.
- État d'exécution découplé de son placement physique.
- Surface d'attaque minimisée par tâche.
- Vérification formelle à terme (ex : noyaux type seL4).

- Le serving agentique = **trajectoires longues et stateful** (modèle, harnais, sandbox).
- **Aries** relie résultats de tâche et comportement système.
- Expériences + traces de production → quatre constats robustes.

Message clé

Optimiser le seul débit de tokens ne résout pas les vrais goulots d'étranglement des agents.

Merci ! Questions ?